SERVICE INTEGRATION CONTRACT

Backend Integration Handover

What the Financial AI service needs from the MyFi Laravel backend — and why each rule exists.

FOR Laravel backend engineer
AI SERVICE Complete & tested
BLOCKED ON The endpoints below
PYTHON NEEDED None

CONTENTS

01 What this service is
02 What you need to build
03 The service token
04 The security rules
05 The endpoints
06 Schema gotchas
07 Errors
08 Conversation storage
09 Proxy for the app
10 Definition of done
11 Questions for you

01 What this service is

A separate Python/FastAPI service that answers users' natural-language questions about their own finances — *"how much did I spend on groceries last month?"* It calls OpenAI.

It does not own any financial data and never writes any. Your Laravel backend remains the single source of truth. The AI service asks you for data, does deterministic arithmetic on it, and asks the model to phrase the answer.

```
Mobile / web app │ (existing user session — unchanged) ▼ Laravel backend
──────────────────────────► AI service ▲ 1. mints a short-lived JWT │ │ 2. serves
/internal/* endpoints │ └──────────────────────────────────────────┘ server-to-server,
private network
```

The AI service is **never** called directly by the mobile app. The app talks to you; you talk to the AI service.

02 What you need to build

1. **A JWT minting step** — assert who the user is (§3)
2. **Eleven read-only `/internal/*` endpoints** — serve their data (§5)
3. **A proxy endpoint** for the app to reach the AI service (§9)

The endpoints are straightforward reads. The care is all in the scoping rules in §4 — please read that section twice.

03 Authentication: the service token

The contract

When your backend calls the AI service, it sends a JWT you sign. That token is the **only** thing that tells the AI service who the user is. It will not accept a user id from a request body, a header, or the user's message text.

```
Authorization: Bearer <jwt>
X-Session-Id: <conversation id>
```

Required claims

CLAIM	REQUIRED	VALUE
`sub`	**yes**	`application_users.id` as a string — see the warning below
`session_id`	yes*	conversation id (`chat_sessions.id`)
`exp`	**yes**	expiry. Tokens without it are **rejected**
`iat`	recommended	issued-at; enables the max-lifetime check
`timezone`	optional	from `application_users.timezone` ; defaults to UTC
`currency`	optional	defaults to USD
`aud` / `iss`	optional	verified only if the AI service is configured with them

* May be supplied via the `X-Session-Id` header instead.

Token lifetime must be ≤ 15 minutes. A validly-signed token with a longer lifetime is rejected. Mint a fresh one per request — they're cheap.

Algorithm **HS256**, shared secret of **at least 32 bytes** (RFC 7518 §3.2). The AI service refuses to start in production with a shorter one. Share it out-of-band.

● CRITICAL · `SUB` MUST BE `APPLICATION_USERS.ID`

Your database has two user tables: `users` (Laravel's default) and `application_users` (the real end users — Plaid tokens, timezone, 2FA).

Every foreign key in the schema that references a user points at `application_users.id` — all 25 of them, including `chat_sessions`, `investment_items` and `plaid_items`. Not one references `users`.

Both tables are `AUTO_INCREMENT` from 1, so `users.id = 5` and `application_users.id = 5` are **different people**. If a token carries the wrong one, the AI service will confidently answer with another user's finances and **nothing will error** — both are plausible integers.

This is the single most dangerous mistake available in this integration. Please add a test for it.

Laravel example

```
// composer require firebase/php-jwt
use Firebase\JWT\JWT;

class AiServiceTokenFactory
{
    public function forUser(ApplicationUser $user, string $sessionId): string
    {
        $now = time();

        return JWT::encode([
            'sub'        => (string) $user->id,   // application_users.id
            'session_id' => $sessionId,
            'timezone'   => $user->timezone ?: 'UTC',
            'currency'   => 'USD',
            'iat'        => $now,
            'exp'        => $now + 300,           // 5 minutes
        ], config('services.ai.jwt_secret'), 'HS256');
    }
}
```

04 The security rules

These four rules are what stop one user seeing another's money. The AI service enforces what it can on its side; these are the parts *only you* can guarantee.

● RULE 1 · VERIFY `USER_ID` YOURSELF

Every endpoint receives `?user_id=…`. **You must independently verify it against the identity in the service token, and reject any mismatch with 403.**

The AI service authenticates to you with a single shared service token — the same one for every user. If you trust the `user_id` parameter without checking, anyone able to reach the AI service with any valid JWT can read any user's finances. This is the largest remaining hole in the system, and it can only be closed here.

```
// app/Http/Middleware/VerifyAiServiceUser.php
public function handle(Request $request, Closure $next)
{
    $claimed  = (string) $request->query('user_id');
    $verified = (string) $request->attributes->get('ai_token_sub');

    abort_if($claimed === '' || $claimed !== $verified, 403, 'user_id mismatch');

    return $next($request);
}
```

Apply it to the whole `/internal` route group.

● RULE 2 · SCOPE EVERY QUERY. NO EXCEPTIONS.

Use a global scope or an explicit `where('user_id', $userId)` on every single query. **Twelve financial tables have no foreign key on `user_id`**, so the database will not catch a mistake:

`plaid_transactions` · `plaid_accounts` · `bills` · `budgets` · `categories` · `liability_accounts` · `liability_credit` · `liability_loans` · `liability_mortgages` · `recurring_transactions` · `investment_snapshots` · `credit_scores`

`plaid_transactions` alone holds ~261,000 rows with no referential integrity on ownership.

● RULE 3 · INVESTMENTS NEED A THREE-LEVEL JOIN

`investment_holdings`, `investment_accounts`, `investment_transactions` and `investment_securities` have **no `user_id` column at all**. Ownership is reachable only through:

```
investment_holdings.account_id → investment_accounts.id
investment_accounts.item_id    → investment_items.id
investment_items.user_id       → application_users.id
```

Someone writing `SELECT * FROM investment_holdings WHERE user_id = ?` gets an error and fixes it. Someone writing it *without* the `WHERE` gets **every user's holdings** and no warning at all.

```
$holdings = DB::table('investment_holdings as h')
    ->join('investment_accounts as a', 'h.account_id', '=', 'a.id')
    ->join('investment_items as i',    'a.item_id',    '=', 'i.id')
    ->where('i.user_id', $userId)                // the only real scope
    ->leftJoin('investment_securities as s', 'h.security_id', '=', 's.security_id')
    ->select('h.id', 'h.quantity', 'h.institution_value', 'h.cost_basis',
             'a.account_id', 's.name', 's.ticker_symbol', 'a.iso_currency_code')
    ->get();
```

Please write a dedicated test that user A cannot see user B's holdings.

● RULE 4 · STAMP `OWNER_USER_ID` ON EVERY RECORD

Add `"owner_user_id": "<the user id>"` to **every** object in every response.

The AI service checks this against the user it asked about and aborts the whole request if any record belongs to someone else. It catches a dropped `WHERE` clause before the data reaches the model or the user. One line per response, and the cheapest insurance in this integration.

05 The endpoints

All are **GET**, read-only, JSON, on your internal base URL. All take `user_id`. All must apply Rules 1–4.

Money is a `{"amount": "…", "currency": "…"}` object. **Send amounts as strings**, not floats — these are financial figures and float rounding is not acceptable. Dates are `YYYY-MM-DD`.

GET `/internal/users/profile?user_id=`

```
{ "user_id": "42", "currency": "USD", "timezone": "America/New_York" }
```

Source: `application_users`.

GET `/internal/accounts?user_id=`

```
[{
  "owner_user_id": "42",
  "account_id": "abc123",
  "name": "Everyday Checking",
  "type": "checking",
  "balance": { "amount": "2401.55", "currency": "USD" },
  "institution_name": "Chase",
  "is_active": true
}]
```

Source: `plaid_accounts`, using `current_balance`. **Note the column `avaliable_balance` is misspelled in the schema** — match it exactly or it won't bind. `type` should be one of checking / savings / credit / investment / loan.

GET

`/internal/transactions?user_id=&start=&end=&category=&merchant=&min_amount=&max_amount=`

Only `user_id` is guaranteed present; apply the others when supplied.

```
[{
  "owner_user_id": "42",
  "transaction_id": "txn_001",
  "account_id": "abc123",
  "amount": "84.10",
  "currency": "USD",
  "date": "2026-08-04",
  "merchant_name": "Walmart",
  "description": "WALMART #1234",
  "category": "Groceries",
  "pending": false
}]
```

RESPONSE FIELD	COLUMN
`transaction_id`	`transaction_id`
`amount`	`amount` — **see §6, do not change the sign**
`date`	`transaction_date`
`merchant_name`	`name`
`category`	join `plaid_categories.display_name` via `plaid_category_id`
`pending`	`pending` is **varchar** — parse to a real boolean

You must filter `is_deleted = 0`. Deleted transactions would otherwise be counted in every total.

TRANSACTION_ID IS NULL FOR MANUAL TRANSACTIONS — RETURN THEM ANYWAY

56 rows in the dump you shared were typed into the app by the user rather than imported from Plaid, and that column is null for every one of them. We found this by running the assistant against your real data: skipping those rows understated one user's July spending by exactly $500, the amount they had entered by hand. Money a user typed in is as real as money Plaid imported.

- Do **not** filter on `transaction_id IS NOT NULL`, and do not join in a way that silently drops those rows.
- Send `plaid_transactions.id` (the primary key) as well, so a row without a Plaid id can still be identified. We use it as `manual-{id}`.

● NO DAY-BY-DAY ENDPOINT IS NEEDED

The assistant answers “how much did I spend each day this week?” and “how much did I spend on August 20?” by grouping the rows this endpoint already returns. Do not build an aggregation endpoint for it.

Two consequences for you:

- `date` must be the real transaction date on every row, not a created-at or imported-at timestamp. A daily figure is only as good as that column.
- `start` and `end` are **inclusive on both ends**. `start=2026-08-20&end=2026-08-20` must return that single day’s transactions, not an empty list — a one-day window is how a specific-date question is asked.

● DECISION NEEDED · `IS_REVIEW_TRANSFER`

Transfers between a user's own accounts should almost certainly be excluded from spending totals, or the assistant will double-count money the user simply moved between their own accounts. Please confirm.

`GET` `/internal/bills?user_id=&start=&end=`

```
[{
  "owner_user_id": "42",
  "bill_id": "bill_9",
  "name": "Electricity",
  "amount": { "amount": "120.00", "currency": "USD" },
  "due_date": "2026-09-01",
  "is_paid": false
}]
```

● DECISION NEEDED · THE `BILLS` TABLE HAS NO BILLS IN IT

Its columns are `total_saving_amount`, `monthly_saving_amount`, `user_id`. There is no name, amount due, due date, or paid flag.

Bill-like data appears to live on `plaid_transactions` (which has `due_date`, `frequency`, `is_recurring`) and/or `plaid_recurring_predictions`. **Please tell us where bills really are.**

Until then, return `[]` — an empty list makes the assistant say "I don't have that", which is correct. Returning savings figures labelled as bills would make it state something false.

GET `/internal/budgets?user_id=&start=&end=`

```
[{
  "owner_user_id": "42",
  "category": "Groceries",
  "budgeted_amount": { "amount": "500.00", "currency": "USD" },
  "spent_amount":    { "amount": "312.40", "currency": "USD" }
}]
```

Source: `budgets` joined to `categories` for the name. Two notes:

- `budgets` has **no period column**, so we assume monthly by convention. Please confirm.
- `spent_amount` is **not stored** — compute it from `plaid_transactions` for the requested period and category.

GET `/internal/subscriptions?user_id=`

```
[{
  "owner_user_id": "42",
  "subscription_id": "stream_77",
  "name": "Netflix",
  "amount": { "amount": "15.99", "currency": "USD" },
  "billing_cycle": "monthly",
  "next_charge_date": "2026-09-14",
  "is_active": true
}]
```

● **DO NOT USE THE `SUBSCRIPTIONS` TABLE**

That table is MyFi's *own* billing plan (`plan_id` , `app_account_token` , `is_trial` , `price`). If you serve it here, a user asking "what subscriptions am I paying for?" gets told about their MyFi plan instead of Netflix and Spotify.

Use **`plaid_recurring_streams`**, which maps cleanly:

RESPONSE FIELD	COLUMN
`subscription_id`	`stream_id`
`name`	`description`
`amount`	`amount`
`billing_cycle`	`frequency`
`next_charge_date`	`next_payment_date`
`is_active`	`active`

GET `/internal/recurring-transactions?user_id=`


```
[{
  "owner_user_id": "42",
  "recurring_id": "stream_31",
  "name": "Gym Membership",
  "amount": { "amount": "45.00", "currency": "USD" },
  "frequency": "monthly",
  "category": "Health",
  "next_expected_date": "2026-09-02"
}]
```


Also from `plaid_recurring_streams`. The `recurring_transactions` table only holds `transaction_id` + `recurring_date`, and its `user_id` is a nullable `int(122)` — not usable as a source here.

GET `/internal/investments?user_id=`


```
[{
  "owner_user_id": "42",
  "holding_id": "hold_5",
  "account_id": "inv_abc",
  "symbol": "VTI",
  "name": "Vanguard Total Stock Market ETF",
  "quantity": "12.5000",
  "market_value": { "amount": "3120.00", "currency": "USD" },
  "cost_basis":   { "amount": "2800.00", "currency": "USD" }
}]
```


Use the three-table join from Rule 3. `market_value` ← `institution_value`; `symbol` ← `investment_securities.ticker_symbol`.

GET `/internal/liabilities?user_id=`

```
[{
  "owner_user_id": "42",
  "liability_id": "liab_3",
  "name": "Chase Sapphire",
  "type": "credit_card",
  "balance": { "amount": "1840.22", "currency": "USD" },
  "interest_rate": "19.99",
  "minimum_payment": { "amount": "45.00", "currency": "USD" }
}]
```

Source: `liability_accounts` as the base, enriched per type:

TYPE	TABLE	INTEREST_RATE	MINIMUM_PAYMENT
`credit_card`	`liability_credit`	from `aprs` JSON	`minimum_payment_amount`
`student_loan` / `auto_loan`	`liability_loans`	`interest_rate_percentage`	—
`mortgage`	`liability_mortgages`	`interest_rate`	—

`minimum_payment` matters: the assistant uses it for debt-to-income ratios.

GET `/internal/financial-summary?user_id=&start=&end=`

```
{
  "owner_user_id": "42",
  "period_start": "2026-08-01",
  "period_end": "2026-08-31",
  "total_income":   { "amount": "5200.00", "currency": "USD" },
  "total_expenses": { "amount": "3145.60", "currency": "USD" },
  "net_cash_flow":  { "amount": "2054.40", "currency": "USD" },
  "income_availability": "available",
  "expenses_availability": "available"
}
```

Both totals must be positive numbers — this endpoint reports magnitudes, unlike the transactions endpoint which passes raw signed amounts through.

> ● **"MISSING" IS NOT THE SAME AS ZERO**
>
> The `*_availability` fields must be `"available"`, `"missing"`, or `"not_applicable"`. Use `"missing"` when you genuinely have no data — do **not** send `0`. Zero means "they earned nothing", and the assistant will say exactly that. `"missing"` makes it say the data isn't available.

GET `/internal/financial-score?user_id=`

```
{
  "owner_user_id": "42",
  "score": 72,
  "score_range_min": 0,
  "score_range_max": 100,
  "factors": ["Strong savings rate", "High credit utilisation"],
  "as_of_date": "2026-08-15"
}
```

Source: the latest `allo_score_snapshots` row (`overall_score`, plus the `*_insight` fields as `factors`). Return `null` if the user has no score — the assistant will never invent one.

06 Schema gotchas that will bite you

● CRITICAL · PASS TRANSACTION SIGNS THROUGH UNCHANGED

In `plaid_transactions`, a **positive** `amount` means money **left** the account (a purchase), and a **negative** amount means money **came in** (a deposit). This is Plaid's own convention.

Verified against 122,326 real rows in the dump:

ROW TYPE	POSITIVE	NEGATIVE
Purchases (Netflix, Uber, Walmart…)	18,300	167
Income (payroll, salary, deposits)	6	7,557

The AI service is configured for exactly this. **Do not "helpfully" flip signs to make them look more intuitive** — if you do, the assistant will report users' salaries as spending. If you have a strong reason to normalize, tell us first: we flip one constant on our side, but both sides must agree explicitly rather than each assuming.

Currency is missing from the schema

`plaid_transactions` and `plaid_accounts` have no currency column, though `investment_accounts` and `liability_accounts` have `iso_currency_code`. The AI service refuses to add up amounts in different currencies — but if everything defaults to `"USD"`, that protection does nothing for a user with non-USD accounts. Send the real currency wherever you have it.

Send money as strings

`{"amount": "84.10"}`, not `{"amount": 84.10}`. JSON floats cannot represent decimal cents exactly. Laravel's `decimal:2` cast produces strings — use it.

07 Errors

Use standard status codes. The AI service maps them to user-facing messages:

YOU RETURN	THE ASSISTANT SAYS
`200` with `[]` or `null`	"I don't have that information"
`403`	request rejected (use for Rule 1 mismatches)
`4xx` / `5xx`	"the service is temporarily unavailable"
timeout (>10s)	"the service timed out"

Prefer `200` with an empty array over an error when a user simply has no data of that kind. An empty list is a fact; an error is a fault.

Keep responses under ~10 seconds. Never include another user's data in an error body.

08 Optional: conversation storage

Right now the AI service keeps chat history in process memory. History is lost on restart and isn't shared between instances — so behind a load balancer, a user's history depends on which instance they hit.

> **YOUR SCHEMA ALREADY HAS MOST OF THE FIX**
>
> `chat_sessions` and `chat_messages` both exist, both scoped to `application_users`, and `chat_messages.role` is already `enum('user','assistant','system')` — an exact match.

8.1 Three columns you must add

A message is not just a role and a body. The AI service stores three extra fields per assistant message, and **conversation memory breaks without them** — not loudly, which is the problem. Everything still returns 200; the assistant just starts asking questions it has already been answered.

COLUMN	TYPE	NULL	WHAT IT IS
`intent`	`varchar(64)`	yes	The classified intent of the turn. Diagnostic.
`is_clarification`	`boolean`	no. default `false`	True when this assistant turn was a clarifying **question**, not an answer.
`date_phrase`	`varchar(64)`	yes	The period the turn was about, as the user phrased it (`"last month"`).

Why each one matters:

- **`is_clarification`** — when the assistant asks "which period do you mean?" and the user replies "yes", that reply is meaningless on its own. The service needs to know the previous turn was a question in order to read the answer against it. Without this column it cannot tell a question from an answer, re-asks, and the user is stuck in a loop with no way out. This was a real observed failure, not a hypothetical.
- **`date_phrase`** — once the assistant has answered for August 1–22, that window is settled. A follow-up like "what about groceries?" names no period and inherits it. Drop this column and every such follow-up becomes another question about the time period.
- **`intent`** — not load-bearing; keep it for support and analytics.

● STORE `DATE_PHRASE` VERBATIM

Do not interpret, normalise, or translate it. It is deliberately the user's own wording — the service resolves it to concrete dates itself, in the user's timezone, on every turn. Resolving it at write time would freeze `"this month"` to whatever month the row was inserted in.

```
Schema::table('chat_messages', function (Blueprint $table) {
    $table->string('intent', 64)->nullable();
    $table->boolean('is_clarification')->default(false);
    $table->string('date_phrase', 64)->nullable();
});
```

8.2 Who owns it

Two options — either works, we just need to know which:

- **You own it:** add `GET/POST/DELETE /internal/conversations/*` endpoints and we implement a store against them.
- **We own it:** give the AI service a read/write connection limited to those two tables.

Every query must be scoped by both `user_id` AND `session_id` — a session id alone is not sufficient, because it would otherwise be guessable across users.

09 The proxy endpoint for the app

The app must not call the AI service directly — it would need the shared secret. Add something like:

```
Route::post('/api/chat', function (Request $request) {
    $user = $request->user();                       // your existing auth
    $sessionId = $request->input('session_id');

    $token = app(AiServiceTokenFactory::class)->forUser($user, $sessionId);

    $response = Http::withToken($token)
        ->withHeaders(['X-Session-Id' => $sessionId])
        ->timeout(30)
        ->post(config('services.ai.base_url') . '/api/v1/chat', [
            'session_id' => $sessionId,             // must match the header
            'message'    => $request->input('message'),
        ]);

    return response()->json($response->json(), $response->status());
})->middleware('auth:sanctum');
```

Two details that will cost you a debugging hour otherwise:

- `session_id` in the body **must equal** `X-Session-Id`. The AI service rejects a mismatch with 422 rather than silently picking one.
- Allow ~30s. A chat turn involves two or three OpenAI calls.

Rate limiting: the AI service already caps requests per user per minute (default 60) and returns `429`. Pass that through to the app.

10 Definition of done

- [ ] Shared JWT secret generated (≥32 bytes) and exchanged securely
- [ ] Token factory mints HS256 tokens with `sub`, `session_id`, `iat`, `exp` ≤ 15 min

- [ ] `sub` is `application_users.id` — with a test proving it
- [ ] Middleware rejects `user_id` ≠ token `sub` with 403 (Rule 1)
- [ ] All 11 endpoints implemented, scoped by `user_id` (Rule 2)
- [ ] Investments scoped via the `investment_items` join (Rule 3)
- [ ] Every record carries `owner_user_id` (Rule 4)
- [ ] `plaid_transactions` filter `is_deleted = 0` on
- [ ] Transaction signs passed through unchanged (§6)
- [ ] Money serialized as strings
- [ ] `financial-summary` total `*_availability` positive; used correctly
- [ ] Subscriptions sourced from `plaid_recurring_streams` not `subscriptions`

- [ ] Decision made on where bills live (§5)
- [ ] Decision made on `is_review_transfer` (§5)
- [ ] Decision made on conversation storage ownership (§8)
- [ ] `intent`, `is_clarification`, `date_phrase` columns added to `chat_messages` (§8.1) — only if conversations move to the database
- [ ] Proxy endpoint added for the mobile app (§9)
- [ ] **Cross-user test: user A's token cannot retrieve user B's data on any endpoint**

That last one is the acceptance test for the whole integration. If it passes on every endpoint, the integration is sound.

11 Questions we need answered

1. Where do **bills** actually live? (§5)
2. Should `is_review_transfer` transactions be excluded from spending? (§5)

3 Are **budgets** monthly? (§5)

4 Who owns **conversation storage**? (§8)

5 Confirm:

`sub` will be `application_users.id` (§3)

6 Confirm: transaction signs passed through unchanged. (§6)

Answers to 1–4 change what gets built. Items 5 and 6 are confirmations we need *in writing*, because getting either wrong produces confident, wrong answers to users rather than visible errors.
